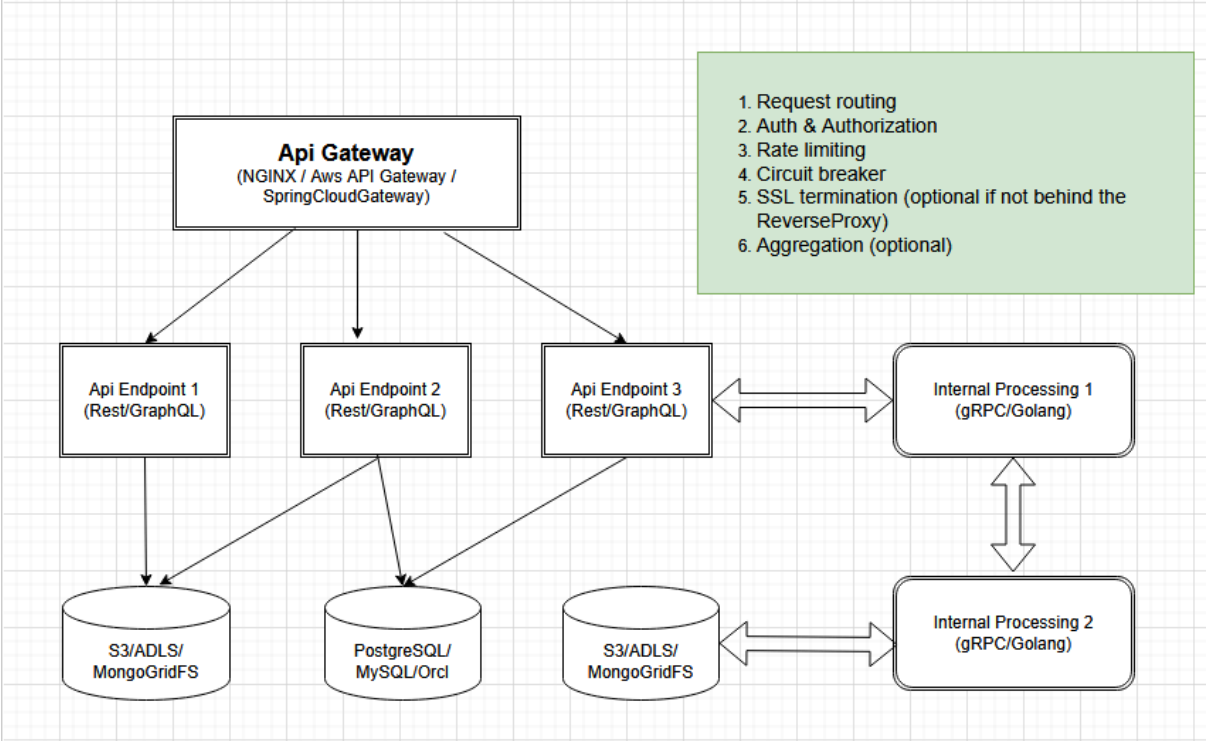
1. Backend Applications Design Guideline

|==========|



1.1 Service Unit App Directory/Package Structure

|==========|

```
com.example.myservice
│
├── MyserviceApplication.java
├── config/              → Configuration classes e.g. filters,securities..
├── controller/          → REST controllers (MVC - C layer)
├── service/             → Business logic   (M layer logic)
│     ├── impl/
├── domain/              → Business Domain
│     ├── entity/        → JPA entities
│     ├── dto/           → Request/Response DTOs
│     ├── mapper/        → MapStruct or manual mappers
│     ├── repository/    → JPA repositories (data access)
├── exception/           → Custom exceptions & handlers
├── utils/                → Utilities
└── client/              → Feign/WebClient clients to other microservices
```

1.2 Basic Coding Principal

|==========|

a. Don’t repeat yourself (DRY)
b. Single responsibility: A class/function must do only one thing and change/update/fix for only one cause or reason.

1.3 Logging Guideline

|==========|

a. Correlation ID Pattern (Critical for Microservices)

```
{
  "timestamp": "2026-02-14T10:00:00Z",
  "service": "order-service",
  "correlationId": "abc-123",
  "message": "Order created"
}
```

b. Structured Logging Pattern

```
{
  "level": "ERROR",
  "service": "payment-service",
  "userId": 123,
  "orderId": 789,
  "errorCode": "CARD_DECLINED"
}
```

c. Prod: INFO, WARN, ERROR, FATAL
d. Dev: All above and DEBUG
e. Async Log Appender [if possible depending on backend frameworks]
f. Firebase for frontend application. [if possible]

2. Api Gateway Design Guideline

|==========|

A. Role of the API Gateway:
   a. Request routing
   b. Authentication & authorization filter
   c. Circuit breaker
   d. Rate limiting
   e. Aggregation (optional)
   f. SSL termination (optional if not behind the reverse-proxy)
B. Common gateway technologies:
   a. NGINX
   b. Amazon API Gateway
   c. SpringCloud Gateway
   d. Apigee

3. Containerization Guideline

|==========|

A. Follow 12-Factor Principles:
   a. Stateless
   b. Disposable
   c. Configured via environment
   d. Logging to stdout/stderr
   e. Backed by external services

B. Service health metrics monitoring:
    a. Actuator for Jvm matrics, convert and port to Prometheus, visualize using Graphana
    b. Add health endpoints: /health, /ready, /live etc
C. Resource Limits & Requests

```
resources:
  requests:
    cpu: "200m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

4. Git Branching and Commit massage patterns

|==========|

A. Branch naming pattern:
    a. <ticket-id>:Short title: Short description
B. Master or Main must be merged via Pull/Merge request
C. After merge via pull request CICD pipeline (inside Github) shall trigger => run unit test => build image => deploy image on the staging server.
D. Production deployment should be manual.
E. Git commenting pattern:
    a. <ticket-id>:<feat | fix | test | chore | doc>: message text … .. …. …..
    b. e.g. STL-121:feat: my test commit for there and other blah blah.

5. Jira/Trello ticketing & boards

|==========|

Todo	In progress	Dev Review (Pull Request)	Testing	Done